

Trainee Software Engineer

Technical Exercise

Shopify-powered marketplace with a local product catalog, Cash on Delivery checkout, and a logged background job queue.

Stack	Next.js (App Router) • Prisma • MySQL • Redis + BullMQ • Shopify Admin GraphQL API
UI theme	AdminLTE v3.2.0 — E-commerce product page (reference in Section 3)
Payment	Cash on Delivery (COD) only — no payment gateway

1. Purpose of This Exercise

This exercise mirrors the kind of work you will do on the team: connecting a web application to a third-party platform, keeping data in sync reliably, and running work in the background in a way that can be monitored and debugged. We are less interested in visual polish than in whether the system is **correct, well structured, and observable**.

By the end, a reviewer should be able to:

- 1 Run a sync that copies products from a Shopify store into your MySQL database.
- 2 Browse those products in a storefront styled with the AdminLTE theme.
- 3 Add products to a cart and place an order using Cash on Delivery.
- 4 See that order appear in Shopify, created by a background job.
- 5 Read clear logs showing every job that ran, how long it took, and whether it succeeded or failed.

2. System Overview

The application has one core rule: **the storefront never reads products from Shopify directly**. Shopify is the source of truth for the catalog, but products are copied into MySQL by a background job, and the storefront only ever reads from MySQL. Orders flow the other way: they are saved locally first, then pushed to Shopify by a background job.

Component	Responsibility
Next.js app	Renders the storefront pages (listing, product detail, cart, checkout, confirmation) and exposes API routes / server actions for cart and order submission.
MySQL + Prisma	Local system of record for products, variants, orders, and job logs. The only data source the storefront reads from.
Job queue (Redis + BullMQ)	Holds background jobs so slow or failure-prone work (Shopify calls) never blocks a user request.

Component	Responsibility
Queue worker	A separate Node process that consumes jobs: syncs products into MySQL and submits orders to Shopify.
Shopify store	Source of product data and destination for COD orders. Accessed only through the official Admin GraphQL API.

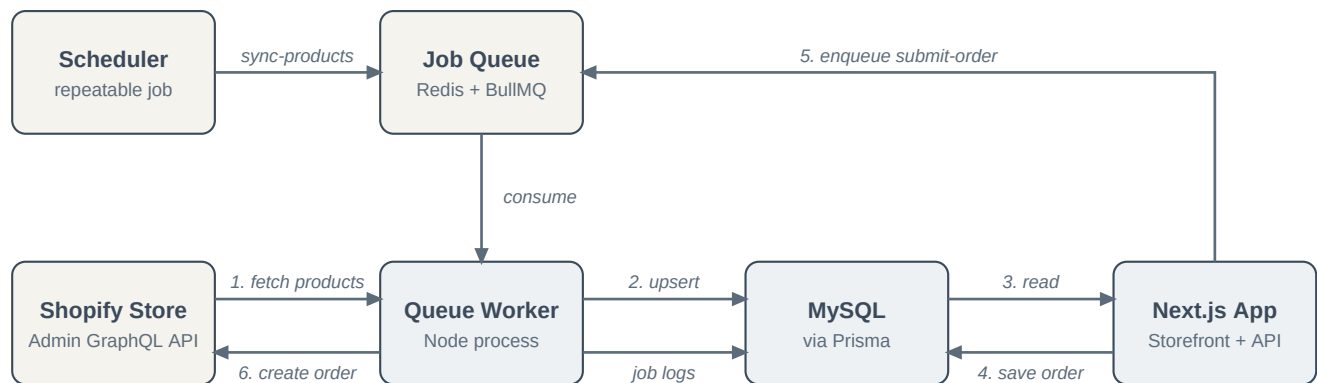


Figure 1 — Data flow. Steps 1-3 are the product sync path; steps 4-6 are the order path.

3. UI Theme — AdminLTE

All storefront screens must be built using the **AdminLTE** theme (Bootstrap-based, open source). Use version **3.2.0**, which contains the E-commerce product page this exercise is based on.

Reference	Link / Path
GitHub repository	https://github.com/ColorlibHQ/AdminLTE
Version / tag	v3.2.0 (https://github.com/ColorlibHQ/AdminLTE/tree/v3.2.0)
Product page source	pages/examples/e-commerce.html
Product page live demo	https://adminlte.io/themes/v3/pages/examples/e-commerce.html
Order confirmation (optional)	pages/examples/invoice.html

How to use the theme

- **Product detail page:** convert `e-commerce.html` into a Next.js page/component. Keep its layout: main image with thumbnail gallery, title, option selectors (e.g. colour/size mapped to Shopify variants), price, the **Add to Cart** button, and the Description tab. Replace all hard-coded content with data from your database. Wishlist, comments and rating tabs can be removed or left static.
- **Product listing page:** AdminLTE has no dedicated listing page, so build a responsive grid of product cards using AdminLTE's card components and the same visual style as the product page.
- **Cart and checkout:** use AdminLTE tables and form components (inputs, buttons, validation states).
- **Order confirmation:** the `invoice.html` layout is a good fit for showing the placed order.

- Convert the markup to proper React/JSX components — do not embed raw HTML pages or rely on jQuery for behaviour that React should own (e.g. the image gallery and variant selection).

4. Functional Requirements

4.1 Product listing

- Show all active products from MySQL: image, title, price, and availability (in stock / sold out).
- Clicking a product opens its detail page. Pagination or “load more” is expected if there are many products.

4.2 Product detail

- Display images, title, description, price, and variant options from the database.
- Selecting an option updates the displayed price/availability for that variant.
- **Add to Cart** adds the selected variant and quantity; disable it for out-of-stock variants.

4.3 Cart

- Add, update quantity, and remove line items; show line totals and a cart subtotal.
- The cart must survive page navigation and refresh (cookie, local storage, or a database-backed cart are all acceptable — explain your choice).
- Prices shown in the cart must come from the database, not from values sent by the browser.

4.4 Checkout (Cash on Delivery)

- Collect customer name, phone number, email (optional), and full delivery address.
- Validate inputs on the server. Cash on Delivery is the only payment method.
- On submit: re-check prices and stock from the database, save the order in MySQL with status `PENDING_SYNC`, enqueue a `submit-order` job, and redirect to the confirmation page **immediately** — the user must not wait for Shopify.

4.5 Order submission to Shopify

- The `submit-order` job creates the order in Shopify through the Admin GraphQL API, marked as unpaid / payment pending with a COD note or tag. Either of these approaches is acceptable: `draftOrderCreate` followed by `draftOrderComplete` (with payment pending), or `orderCreate` with a pending financial status.
- On success, store the Shopify order ID on the local order and set its status to `SYNCED`. On final failure, set it to `FAILED` with the error.
- The job must be **idempotent**: a retry must never create a duplicate order in Shopify.

5. Product Sync: Shopify to MySQL

The sync keeps the local catalog in line with Shopify. It runs as a background job, never inside a page request.

- **Schedule**: run `sync-products` as a repeatable job (e.g. every 10-15 minutes), and provide a way to trigger it manually (a CLI script or an admin API route).
- **Pagination**: fetch products using cursor-based pagination so it works for stores of any size.

- **Fields:** title, handle, description, status, images, variants (title, options, price, SKU, inventory).
- **Upsert:** match records by Shopify ID; create new products and update changed ones.
- **Removals:** products deleted or archived in Shopify must no longer appear on the storefront (soft-delete or mark inactive).
- **Rate limits:** respect Shopify's API cost limits; back off and retry when throttled.
- **Bonus:** handle products/create, products/update and products/delete webhooks by enqueueing a job for the single affected product (verify the webhook HMAC signature).

6. Background Jobs and Queue Logging

Use a Redis-backed queue (BullMQ is recommended). The worker should run as its own process, separate from the Next.js server, so either one can restart without affecting the other.

6.1 Job types

Job	Triggered by	What it does	Retry policy
sync-products	Schedule, manual trigger	Pulls all products from Shopify and upserts into MySQL.	3 attempts, exponential backoff
sync-product (bonus)	Shopify webhook	Syncs one product by its Shopify ID.	3 attempts, exponential backoff
submit-order	Checkout submission	Creates the COD order in Shopify and updates the local order.	5 attempts, exponential backoff; then marked FAILED

6.2 Logging requirements

Logging is a **graded requirement**, not an extra. When something goes wrong, a developer should be able to answer “what happened, to which record, and why” from the logs alone.

- Use a structured logger (Pino or Winston) that outputs JSON. Do not use bare `console.log`.
- Every job logs when it **starts**, meaningful **progress** (e.g. page 3 of products, 50 upserted), and when it **completes** or **fails**.
- Each log line includes: job name, job ID, attempt number, related entity ID (order ID / product ID), and on completion the duration in milliseconds.
- Failures log the error message and stack trace, and whether another retry is scheduled.
- Listen to queue-level events (`completed`, `failed`, `stalled`) and log them.
- Persist a summary of each job run to a `JobLog` table so history survives restarts.
- Never log secrets (API tokens) or full customer details — mask phone numbers and addresses.

Example log output:

```
{ "level": "info", "job": "submit-order", "jobId": "42", "attempt": 1, "orderId": "ord_118",  
  "msg": "Creating order in Shopify" }  
{ "level": "error", "job": "submit-order", "jobId": "42", "attempt": 1, "orderId": "ord_118",  
  "err": "THROTTLED", "retryInMs": 2000, "msg": "Shopify request failed, retry scheduled" }  
{ "level": "info", "job": "submit-order", "jobId": "42", "attempt": 2, "orderId": "ord_118",  
  "shopifyOrderId": "gid://shopify/Order/5512", "durationMs": 812, "msg": "Order synced" }
```

Recommended

Add a queue dashboard such as Bull Board, mounted on an admin-only route, so reviewers can inspect waiting, active, completed and failed jobs visually.

7. Data Model

A suggested starting point. You may change it, but explain any significant changes in your README.

Model	Key fields	Notes
Product	id, shopifyId (unique), title, handle, description, status, isActive, syncedAt	One row per Shopify product.
ProductImage	id, productId, url, altText, position	Ordered image gallery.
Variant	id, productId, shopifyVariantId (unique), title, options, sku, price, inventoryQty	What the customer actually adds to the cart.
Order	id, shopifyOrderId (nullable), customerName, phone, email, address fields, subtotal, paymentMethod = COD, status, lastError, createdAt	Status: PENDING_SYNC, then SYNCED or FAILED.
OrderItem	id, orderId, variantId, quantity, unitPrice	Price captured at time of order.
JobLog	id, jobName, jobId, status, attempt, entityId, durationMs, error, startedAt, finishedAt	History of every job run.

Store money as `Decimal`, not floating-point numbers.

8. Suggested Project Structure

This is a guide, not a requirement. What matters is a clear separation between the web app, the worker, and shared code.

```
marketplace/
  prisma/
    schema.prisma      # data model + migrations
  src/
    app/                # pages: /, /products/[handle], /cart, /checkout, /orders/[id]
    components/         # AdminLTE-based UI (ProductCard, ProductGallery, ...)
    lib/
      db.ts             # Prisma client
      shopify.ts        # Shopify GraphQL client
      logger.ts         # structured logger
      queue.ts          # queues + job definitions (shared)
    jobs/
      sync-products.ts
      submit-order.ts
  worker/
    index.ts            # starts worker + repeatable sync
  docker-compose.yml    # MySQL + Redis for local development
  .env.example
  README.md
```

9. Technical Requirements

- Next.js with the App Router and TypeScript.
- Prisma as the only database access layer, targeting MySQL. Include migrations.
- Redis-backed queue (BullMQ recommended) with the worker running as a separate process.
- Shopify access only through the official Admin GraphQL API, using a development store and a custom app access token.
- All configuration via environment variables; provide `.env.example` and never commit real credentials.
- Server-side validation on checkout (e.g. Zod) and consistent error handling across API routes and jobs.
- A `docker-compose.yml` for MySQL and Redis is strongly encouraged so reviewers can run the project quickly.

10. Deliverables

- **Source code** in a Git repository with a meaningful commit history (not one final commit).
- **README** covering: prerequisites, environment variables, how to start MySQL/Redis, run migrations, start the web app, start the worker, and trigger a manual product sync.
- **Design note** (half a page to one page): your queue design, sync strategy, how you prevent duplicate orders, and trade-offs or shortcuts you would fix with more time.
- **Screenshots or a short screen recording** of the full flow: sync, browse, add to cart, checkout, and the order in Shopify.

11. Evaluation Criteria

Area	What we look for
Functionality	Sync, listing, product page, cart, and COD checkout all work end to end; orders reach Shopify.
Architecture	Clear separation of web app, worker, and shared code. Storefront reads only from MySQL.
Reliability	Retries with backoff, idempotent order submission, correct handling of deleted products and API throttling.
Logging & observability	Structured, consistent, useful job logs; job history persisted; failures easy to diagnose.
Code quality	Readable, typed, well-named code; sensible Prisma schema; no duplicated logic.
UI implementation	Faithful use of the AdminLTE product page; clean conversion to React components.
Documentation	A reviewer can run the project from the README without asking questions.

12. Bonus (Optional)

- Shopify webhooks for real-time product updates, with HMAC verification.
- Admin page listing orders and their sync status, with a “retry” action for failed orders.
- Unit tests for the sync mapping and order job; an integration test for checkout.
- Search and category/collection filtering on the listing page.